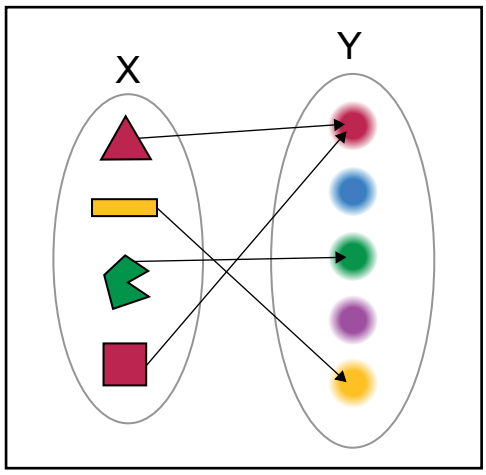

Functions in functional languages try to model mathematics. So while two different inputs can gave the same output (square of both -2 and 2 is 4), a single input should not have multiple possible outputs.

the same. Rather than provide a rate each time, we can supply that once and use the result. Let's see it in action:

```
let simple_interest rate time principal =
   principal * time * rate / 100

let si_4pc = simple_interest 4

let account1 = si_4pc 3 1000
let account2 = si_4pc 2 3000
```

In the above code sample, we create this simple interest function and supply only one parameter. This returns a function that is the simple_interest function with a rate of 4 per cent applied. Then we can use this new function si_4pc to calculate the interest for different time periods and principal amounts.

Another brilliant thing about F# functions is how easy they are to combine. In most languages if you had to take a value and apply multiple operations to it in series you would do something like:

```
result = sum( range( square( double( x ) ) ) )
```

Now this can be a little confusing since the operators appear in the opposite order in which the data is flowing. First x is doubled, then squared and so on. In F#, the same is done in this manner:

```
let result = x |> double |>
square |> range |> sum
```

Here F# will pass x as a value to double, pass the result of that to square, the result of that to range and finally to sum.

If you wanted to make a new function that combined these functions you could do that as simply as:

```
let doubleSquareRangeSum = double
>> square >> range >> sum
```

C#* pointers

>>Interesting developer news

*Introduction to Functional Programming

>>If you're interested in functional programming, this course at EDX.org should get you started.

http://dgit.in/EDXCourse

*F# and Xamarin for Mobile

>>This video both introduces F# and also shows how to use it to create Android and iOS apps.

http://dgit.in/XamarinApps

*F# for Web Apps

>>If you're wondering how to server-side web apps using F#, let this video teach you how.

http://dgit.in/WebDeveApps

*New features of F#4.0

>>Here's a quick roundup of six new features of F# 4.0 that you'd be interested in.

http://dgit.in/QuickFSharp4

Interactive page

Records

While records are quite common in other programming languages, A record in F# is similar to a struct in C / C++; here is how to make one:

```
type Point = {x:int; y:int}
```

Here, we declare a new Point type that has two fields, x and y. The great thing now is that in F#, any structure you create that matches this pattern will automatically be a Point! For instance:

```
let p1 = {x1=11; y1=12}
```

This is automatically a Point, despite the fact that you never explicitly declare it as one. You can confirm this by typing it in the interpreter.

You can access these fields as you'd expect with p1.x1 and p1.y1 but F# has a better way:

```
let {x=x1; y=y1} = p1
```

This will assign the x field of p1 to x1 and the y field of p1 to y1. You could use this as part of a function, for example:

```
let sum_coords {x=x1; y=y1} =
   x1 + y1
```

This function takes a point, and sums its x and y ordinates.

Discriminated Unions

Discriminated Unions are something you will only find in functional languages. They are an great way to define complex types that combine multiple other types.

For instance, you might define a Colour type as follows:

```
type Colour =
   | Red
   | Green
   | Blue
   | Other of string
```

Here, we are defining Colour as a new type that can either be Red, Green, Blue, or Other. In case it is Other we can provide our own value. All of the following are of type Colour:

```
let g = Green
let cyan = Other "cyan"
```

Let's make this Colour type more interesting by allowing for combinations of colours:

```
type Colour =
   | Red
   | Green
   | Blue
   | Other of string
   | Combination of Colour * Colour
```

With the above we can define colours as:

```
let colour1 = Combination(Red, Blue)
let colour2 = Combination(colour1, Green)
```

The Colour * Colour above allows use to make a pair of Colours, as you can see, we have a recursive type here where a colour can be a combination of colours!